

Programming Language Principles, Design, and Implementation

Compilers - Runtime Environments

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Runtime Environments

- ▶ Allocation of storage locations
- ▶ Access of variables and data
- ▶ Heap management

Further reading:

- ▶ Chapter 7 of the Dragon book
- ▶ Chapter 6 of “Modern Compiler Implementation”

Problem

A compiler must correctly map the high-level constructs of the source language to the low-level target machine constructs.

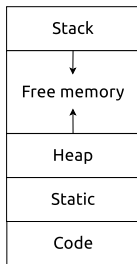
To do so, it makes use of a **runtime environment**, which is a collection of data structures maintained at runtime, such as the stack and the heap.

Storage Organization

The target code runs in its own **logical address space**, shared between the compiler, OS, and target machine

The OS maps the **logical addresses** into **physical addresses**. The virtual memory is typically divided into the following **program** and **data** areas:

where



- ▶ the generated target code is computed at compile time and stored in **Code**
- ▶ some objects (e.g., global constants) are also fixed at compile time and stored in **Static**
- ▶ the dynamic **Stack** stores **activation records** generated during procedure calls, and starts from a higher address and grows downward
- ▶ the dynamic **Heap** stores dynamically allocated data (e.g., using `malloc` in C), and starts from a lower address and grows upward

Storage Organization

A storage unit is a byte. The runtime storage is assumed to come in blocks of contiguous bytes.

Variables are assigned types during type checking, from which can be derived the number of memory units to store the variables (e.g., 4 memory units to store an integer).

Objects that span over multiple bytes (e.g., characters, integers, floats, arrays), are stored in consecutive bytes and are assigned the address of the first byte. Storage for objects of aggregate types such as arrays, need to be allocated enough memory to hold them.

Stack vs. Heap:

- ▶ a local variable is dynamically allocated on the **stack** within an activation record generated when the procedure containing the variable is called, and removed from the stack when the procedure returns
- ▶ some variables are dynamically allocated to the **heap** if they may outlive a procedure call, and de-allocated, e.g., when manually freed, or when automatically garbage collected.

Activation Records

The stack is usually divided into blocks, called **activation records** or **stack frames**, which are

- ▶ pushed onto the stack every time a function is called,
- ▶ and popped when the function returns.

It holds: the local variables, parameters, return addresses, and other temporaries (some of those are stored in registers for efficiency).

A **stack pointer** (stored in a register) identifies where the current stack frame is on the stack.

The activations (i.e., calls) of procedures can be organized in **an activation tree** as follows, showing the calls in the order they happen:

```
1 int fact(int n) {  
2   if (n <= 2) { return n; }  
3   else { return n*fact(n-1); }  
4 }  
5 int main() { return fact(4); }
```

(example in C)

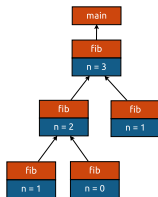


Activation Records

What is the activation tree of the following program:

```
1 int fib(int n) {  
2   if (n < 2) { return 1; }  
3   else { return fib(n-1) + fib(n-2); }  
4 }  
5 int main() { return fib(3); }
```

(example in C)



Activation Records

The stack contains at any point in time a path from the root to a node.

Each node finishes before the node on its right begins.

Given an edge in the activation tree, the parent node is called the **caller**, and the child is called the **callee**.

- ▶ The code executed (called **calling sequence**) to push activation records onto the stack can be divided between caller code, and callee code
- ▶ and similarly for the code executed (called **return sequence**) to pop activation records from the stack.

The activation record of the callee contains a **control link** pointing to the activation record of the caller

Activation Records

Layout of activation records:

- ▶ data communicated between caller and callee (e.g., parameters, return address, return value) is placed at the bottom of the callee's activation record, i.e., top of the caller's activation record, so that they know where to place/access it
- ▶ fixed-length variables are placed in the middle
- ▶ variables for which the size is unknown at this stage (e.g., a dynamically sized array) are placed at the end

In a calling sequence,

- ▶ the caller can store the parameters and return address (value of the program counter, to which the callee must return) onto the stack, and then give control to the callee
- ▶ the callee saves the register values and other status information
- ▶ the callee initializes its local data, and begins execution

In a return sequence,

- ▶ the callee can store the return value
- ▶ the callee restores the old register values
- ▶ the callee jumps to the return address

Activation Records

We said that objects of sizes **unknown at compile time** are allocated onto the **heap**. However, it is sometimes possible to place local variables of unknown size onto the **stack** instead for **efficiency**.

For example, pointers to the first elements of arrays can be stored in the stack frames.

We so far covered local variables, but what about non-local data?

We won't deal with programming languages that allow nested functions here. For those that don't, only global variables are non-local.

This is not true if nested functions are allowed (in Haskell):

```
1 f :: Int -> Int -> Int
2 f x y = g x
3   where
4     g x = if x == 0 then y else g (x - 1)
```

The nested function **g** refers to the variable **y**, which is not local to **g**, and therefore not stored in its stack frame.

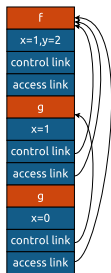
Activation Records

How does `g` get access to `y` in the following program?

```
1 f :: Int -> Int -> Int
2 f x y = g x
3   where
4     g x = if x == 0 then y else g (x - 1)
```

Solution: store a link to the closest function surrounding `g`: this is called an **access link**.

Example:



- ▶ `f 1 2` calls `g 1`, generating a stack frame
- ▶ `g 1` calls `g 0`, generating a stack frame
- ▶ `g 1`'s control link points to `f 1 2`
- ▶ `g 0`'s control link points to `g 1`
- ▶ both `g 1`'s and `g 0`'s access links point to `f 1 2`

Activation Records

This is the simplest case. More complicated cases are when procedure names are passed as parameters, and calls can only be resolved at runtime. We won't cover this here.

See for example Section 7.3.7 of the Dragon book.

Heap management

Some languages (e.g., C and C++), allow creating data that is not tied to the activation record of the procedure call that created it:

- ▶ some memory can be manually allocated using `new`
- ▶ that memory can be manually freed `free`
- ▶ it can also be automatically deallocated by a **garbage collector**

The **memory manager** keeps track of the free memory.

When a program requests some memory:

- ▶ the memory manager tries to produce some contiguous chunk of free heap memory
- ▶ if that is not possible, it requests more memory from the OS
- ▶ if that is not possible, an error is returned

When a program frees some memory:

- ▶ the memory manager returns that memory to the pool of free memory
- ▶ if the OS increased the memory, it is not returned to the OS at this point

Heap management

Desired properties of a memory manager:

- ▶ **space efficiency**: minimize heap space used
- ▶ **program efficiency**: make good use of memory to allow programs to run fast
- ▶ **low overhead**: minimize time spent (de-)allocating memory

Therefore, memory management should be done with an awareness of how memory behaves. Storage is arranged as a **memory hierarchy**, with **faster but smaller** memory closer to the processor, and larger but slower memory further away (oldish data):

kind	size	speed (cycles)
registers	8-64bits	0
L1/L2/L3 caches	KB-MB	1-40
RAM	GB	50-100
SSD	GB-TB	300k

Only registers are managed by the compiler code

Heap management

Further considerations when building a memory manager:

Exploiting locality. Try to place basic blocks that are likely to follow each other on the same page or cache line.

- ▶ The most common instructions and data are placed in the fast/small storage.
- ▶ Data is transferred as blocks of continuous storage (**cache lines/pages**).

Reduce fragmentation. Memory allocations and deallocations lead to a fragmentation of the memory, i.e., large number of non-contiguous free chunks of memory.

Garbage collection. Avoid memory-leak errors by using a garbage collector (not for languages that allow arbitrary pointer arithmetic).

- ▶ **mark-and-sweep**: mark all the unreachable objects, and sweep the entire heap to free them up
- ▶ **mark-and-compact**: move reachable objects to eliminate memory fragmentation
- ▶ ...

Conclusion

What did we cover today?

- ▶ Runtime Environments
- ▶ Allocation of storage locations
- ▶ Access of variables and data
- ▶ Heap management

Next time?

- ▶ Code generation